

TUẦN 1: Java Web Basics - Tài Liệu Lý Thuyết Chi Tiết

Giáo viên hướng dẫn: Senior Java Developer Cấp độ: Người mới bắt đầu Thời lượng: 1 tuần (7 phiên)

Nội dung Tuần 1

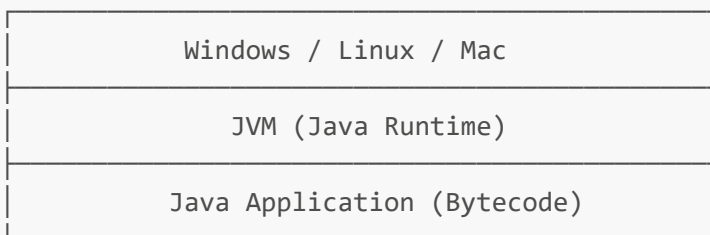
- **Session 1:** Java Web Overview
 - **Session 2:** HTTP & Web Concepts
 - **Session 3:** Servlet Basics
 - **Session 4:** Servlet Lifecycle
 - **Session 5:** Session & Cookies
 - **Session 6:** Deploy First Servlet
 - **Session 7:** Mini Project
-

SESSION 1: Java Web Overview

1.1 JVM, Java EE Basics

JVM (Java Virtual Machine) - Máy Ảo Java

Khái niệm: JVM là một máy ảo cho phép chạy bytecode Java trên bất kỳ hệ điều hành nào. Đây là lý do Java nổi tiếng với khẩu hiệu: **"Viết một lần, chạy ở mọi nơi" (Write Once, Run Anywhere - WORA).**



Quá trình biên dịch và thực thi:

```
Mã Java (.java)
    ↓ [Compiler - javac]
Bytecode (.class)
    ↓ [JVM - Java Virtual Machine]
Mã máy cụ thể (Native Code)
    ↓
Chạy trên hệ điều hành
```

Java EE (Java Enterprise Edition) - Phiên bản Enterprise

Phân loại Java:

- **Java SE (Standard Edition):** Lập trình cơ bản, desktop
- **Java EE (Enterprise Edition):** Ứng dụng Web, doanh nghiệp, máy chủ
- **Java ME (Micro Edition):** Ứng dụng di động (cũ)

Java EE bao gồm:

- Servlet & JSP (tạo Web application)
- JDBC (kết nối cơ sở dữ liệu)
- JMS (gửi tin nhắn)
- RMI (gọi hàm từ xa)
- EJB (lập trình OOP)
- JPA/Hibernate (ORM - Quản lý dữ liệu)

1.2 Servlet Container (Tomcat)

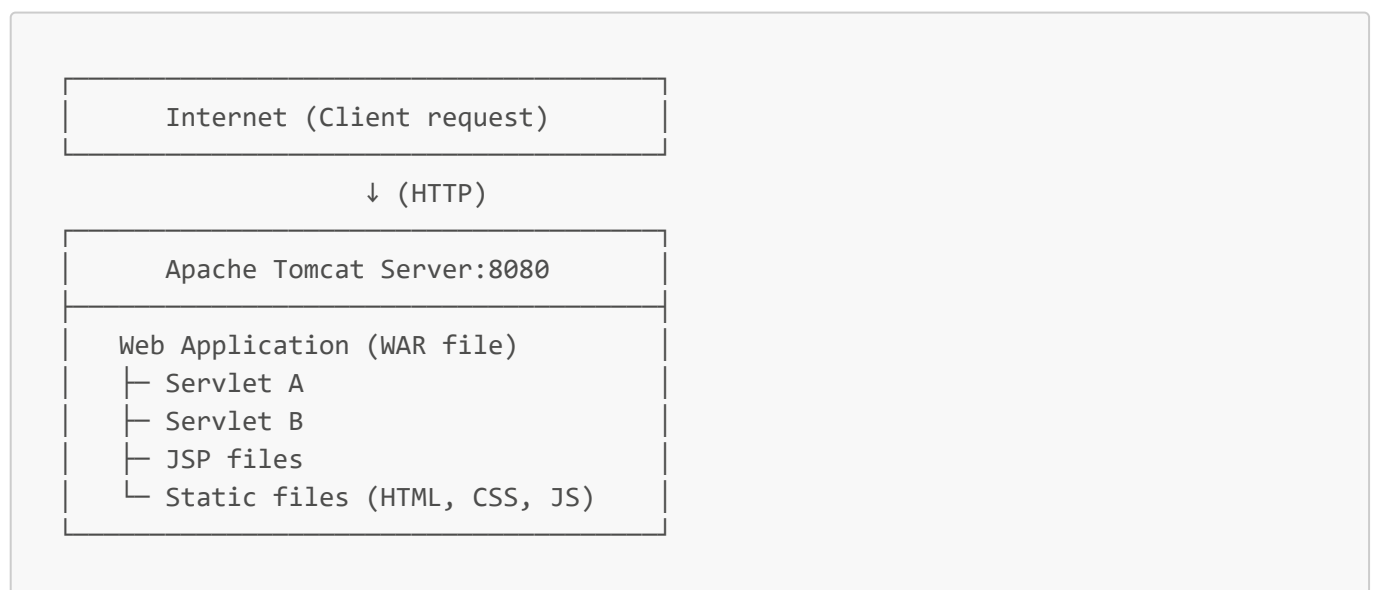
Servlet Container là gì?

Định nghĩa: Servlet Container (cũng gọi là Web Container) là chương trình chạy trên máy chủ, tiếp nhận yêu cầu HTTP từ client, xử lý qua Servlet, và gửi lại phản hồi.

Một số Servlet Container nổi tiếng:

- Apache Tomcat (phổ biến nhất)
- JBoss/WildFly
- GlassFish
- Jetty

Apache Tomcat

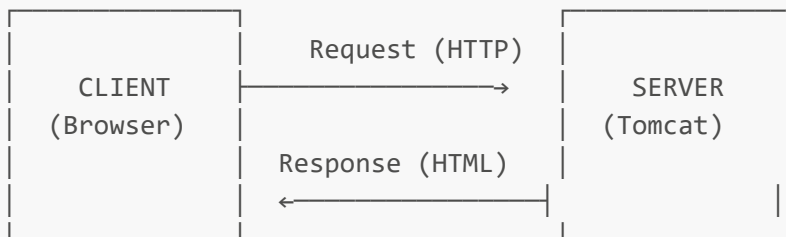


Tại sao dùng Tomcat?

- ☒ Miễn phí, mã nguồn mở
- ☒ Nhẹ, dễ cài đặt
- ☒ Hỗ trợ Servlet & JSP
- ☒ Deploy dễ dàng (file .war)
- ☒ Cộng đồng lớn, tài liệu nhiều

1.3 Client-Server Model & HTTP Basics

Mô hình Client-Server



Quy trình:

1. Client (trình duyệt) gửi HTTP Request đến Server
2. Server (Tomcat) nhận request, xử lý
3. Server gửi HTTP Response (HTML, JSON, v.v.) về cho Client
4. Client hiển thị kết quả

HTTP (HyperText Transfer Protocol)

HTTP là gì? Giao thức truyền tải siêu văn bản - là nền tảng của Web.

Đặc điểm:

- **Textual:** Dữ liệu dạng text, dễ đọc
- **Stateless:** Mỗi request độc lập, không nhớ trạng thái trước
- **Request-Response:** Client khởi động, Server phản hồi

Cấu trúc HTTP Request:

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

```
[Dòng trạng thái]
[Headers]
[Dòng trống]
[Body - nếu có]
```

Cấu trúc HTTP Response:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1234

<html>
  <body>Hello World</body>
</html>

[Dòng trạng thái]
[Headers]
[Dòng trống]
[Body - nội dung]
```

 Tóm tắt Session 1

Khái niệm	Giải thích
JVM	Máy ảo chạy bytecode Java, giúp chạy trên mọi OS
Java EE	Phiên bản của Java cho lập trình Web & doanh nghiệp
Servlet Container	Chương trình xử lý request HTTP và gọi Servlet
Apache Tomcat	Web server phổ biến chạy ứng dụng Java Web
Client-Server	Mô hình: Client gửi request → Server xử lý → gửi response
HTTP	Giao thức truyền tải Web, dạng text, không nhớ trạng thái

 SESSION 2: HTTP & Web Concepts

2.1 HTTP Methods: GET vs POST

GET Method

Định nghĩa: Yêu cầu lấy dữ liệu từ server.

Đặc điểm:

- Dữ liệu gửi qua URL (Query String)
- Giới hạn độ dài (~2000 ký tự)
- Không an toàn (dữ liệu hiển thị trong URL)
- Có thể cache
- Không có body trong request

Ví dụ:

```
GET /search?keyword=java&page=1 HTTP/1.1
Host: www.google.com

URL: https://www.google.com/search?keyword=java&page=1
      ↑
      Query String (tham số)
```

Khi nào dùng GET:

- Lấy dữ liệu (không thay đổi server)
- Tìm kiếm
- Lọc danh sách
- Phân trang

POST Method

Định nghĩa: Gửi dữ liệu để server xử lý/lưu trữ.

Đặc điểm:

- Dữ liệu gửi trong body (không hiển thị URL)
- Không giới hạn độ dài
- An toàn hơn GET
- Không cache (mỗi lần gửi, server xử lý)
- Có body trong request

Ví dụ:

```
POST /login HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 32

username=john&password=secret123
      ↑
      Body (không hiển thị trong URL)
```

Khi nào dùng POST:

- Gửi form đăng nhập
- Tạo dữ liệu mới
- Cập nhật thông tin nhạy cảm
- Upload file

So sánh GET vs POST

Yếu tố	GET	POST
--------	-----	------

Yếu tố	GET	POST
Nơi gửi dữ liệu	URL (Query String)	Body
An toàn	Thấp (dữ liệu hiển thị)	Cao hơn
Giới hạn độ dài	~2000 ký tự	Không giới hạn
Cache	Có	Không
Bookmark	Có	Không
Mục đích	Lấy dữ liệu	Gửi/Lưu dữ liệu

2.2 HTTP Status Codes

Status codes cho biết kết quả của request.

2xx - Thành công

Code	Ý nghĩa	Giải thích
200	OK	Request thành công, server gửi dữ liệu
201	Created	Tạo tài nguyên mới thành công
204	No Content	Thành công nhưng không có dữ liệu trả về

3xx - Chuyển hướng

Code	Ý nghĩa	Giải thích
301	Moved Permanently	Tài nguyên chuyển địa chỉ vĩnh viễn
302	Found	Tài nguyên tạm thời chuyển địa chỉ
304	Not Modified	Dùng phiên bản cache

4xx - Lỗi Client

Code	Ý nghĩa	Giải thích
400	Bad Request	Request sai định dạng
401	Unauthorized	Cần xác thực (chưa đăng nhập)
403	Forbidden	Không có quyền truy cập
404	Not Found	Không tìm thấy tài nguyên

5xx - Lỗi Server

Code	Ý nghĩa	Giải thích
------	---------	------------

Code	Ý nghĩa	Giải thích
500	Internal Server Error	Lỗi trong server
502	Bad Gateway	Gateway lỗi
503	Service Unavailable	Server tạm thời không hoạt động

Ví dụ trong Java:

```
// Gửi status code 200 (thành công)
response.setStatus(HttpServletResponse.SC_OK);

// Gửi status code 404 (không tìm thấy)
response.setStatus(HttpServletResponse.SC_NOT_FOUND);

// Gửi status code 500 (lỗi server)
response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
```

2.3 Request/Response Lifecycle

Quy trình từng bước



Chi tiết từng bước

Bước 1: Client gửi request

```
GET /hello HTTP/1.1
Host: localhost:8080
```

Bước 2-3: Tomcat xử lý

- Tomcat nhận request
- Kiểm tra URL `/hello`
- Tìm Servlet được ánh xạ với `/hello`

Bước 4-5: Servlet xử lý

```
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response) {
        // Xử lý logic tại đây
    }
}
```

Bước 6: Servlet viết response

```
response.setContentType("text/html; charset=UTF-8");
PrintWriter out = response.getWriter();
out.println("<h1>Xin chào</h1>");
```

Bước 7-8: Gửi response

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 25

<h1>Xin chào</h1>
```



 Tóm tắt Session 2

Khái niệm	Chi tiết
GET	Lấy dữ liệu qua URL, an toàn thấp, giới hạn độ dài
POST	Gửi dữ liệu qua body, an toàn hơn, không giới hạn độ dài
2xx Status	Thành công (200 OK, 201 Created)
3xx Status	Chuyển hướng (301, 302, 304)
4xx Status	Lỗi client (404 Not Found, 401 Unauthorized)
5xx Status	Lỗi server (500 Internal Server Error)



SESSION 3: Servlet Basics

3.1 Servlet là gì?

Định nghĩa: Servlet là một class Java chạy trên server, tiếp nhận HTTP Request từ client, xử lý, và gửi lại HTTP Response.

Servlet = Server + Applet (ứng dụng nhỏ chạy trên server)

Vòng đời Servlet

```
Servlet không tồn tại
    ↓
Client gửi request
    ↓
Tomcat tạo object Servlet
    ↓
Tomcat gọi init() - khởi tạo
    ↓
Tomcat gọi service() - xử lý request
    ↓
Tomcat gửi response về Client
    ↓
[Servlet vẫn còn trong memory]
    ↓
Client gửi request khác
    ↓
Tomcat gọi service() lại (không tạo object mới)
    ↓
... (Servlet sẽ sống lâu, có thể xử lý hàng ngàn request)
    ↓
Khi Tomcat shutdown hoặc deploy lại
    ↓
Tomcat gọi destroy()
    ↓
Servlet không tồn tại
```

3.2 HttpServlet - Class cơ bản

HttpServlet là class cha cho tất cả Servlet xử lý HTTP.

Cấu trúc cơ bản

```
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.PrintWriter;
```

```
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response) {
        // Xử lý request GET
    }

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) {
        // Xử lý request POST
    }
}
```

Phương thức doGet()

Khi nào gọi: Client gửi GET request

Ví dụ:

```
protected void doGet(HttpServletRequest request,
                     HttpServletResponse response)
    throws ServletException, IOException {

    // 1. Set kiểu dữ liệu và mã hóa
    response.setContentType("text/html; charset=UTF-8");

    // 2. Lấy out stream
    PrintWriter out = response.getWriter();

    // 3. Viết dữ liệu
    out.println("<h1>Hello World</h1>");
    out.println("<p>Đây là servlet đầu tiên của tôi</p>");

    // 4. Đóng stream
    out.close();
}
```

Giải thích:

- `response.setContentType()`: Báo cho browser biết nội dung là HTML
- `response.getWriter()`: Lấy output stream để viết dữ liệu
- `out.println()`: Viết dữ liệu (tự động thêm dòng mới)

Phương thức doPost()

Khi nào gọi: Client gửi POST request (thường từ form)

Ví dụ:

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    // Lấy dữ liệu từ form
    String username = request.getParameter("username");
    String password = request.getParameter("password");

    // Xử lý (kiểm tra, lưu DB, v.v.)
    if ("admin".equals(username) && "123".equals(password)) {
        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h1>Đăng nhập thành công</h1>");
        out.close();
    } else {
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED,
                           "Tên đăng nhập hoặc mật khẩu sai");
    }
}
```

3.3 Request & Response Objects

HttpServletRequest

Request object chứa thông tin về HTTP Request từ client.

Các phương thức thường dùng:

```
// Lấy tham số từ query string hoặc form body
String username = request.getParameter("username");
String[] interests = request.getParameterValues("interest");

// Lấy all parameters
Enumeration<String> params = request.getParameterNames();

// Lấy thông tin về request
String method = request.getMethod();           // GET, POST
String requestURI = request.getRequestURI();    // /app/login
String queryString = request.getQueryString(); // username=john&age=25
String serverName = request.getServerName();    // localhost
int serverPort = request.getServerPort();       // 8080

// Lấy header
String userAgent = request.getHeader("User-Agent");
String acceptLanguage = request.getHeader("Accept-Language");

// Lấy session
HttpSession session = request.getSession();
```

```
// Lấy IP client
String clientIP = request.getRemoteAddr();

// Lấy cookies
Cookie[] cookies = request.getCookies();
```

Ví dụ thực tế:

```
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {

    String keyword = request.getParameter("q");
    String page = request.getParameter("page");

    // Nếu không có tham số, mặc định
    if (keyword == null) keyword = "";
    if (page == null) page = "1";

    response.setContentType("text/html; charset=UTF-8");
    PrintWriter out = response.getWriter();
    out.println("<h1>Tìm kiếm: " + keyword + "</h1>");
    out.println("<p>Trang: " + page + "</p>");
    out.close();
}
```

HttpServletResponse

Response object để gửi dữ liệu về cho client.

Các phương thức thường dùng:

```
// Set loại nội dung
response.setContentType("text/html; charset=UTF-8");
response.setContentType("application/json; charset=UTF-8");

// Gửi status code
response.setStatus(HttpServletResponse.SC_OK);           // 200
response.setStatus(HttpServletResponse.SC_NOT_FOUND);   // 404
response.setStatus(HttpServletResponse.SC_UNAUTHORIZED); // 401

// Gửi error
response.sendError(HttpServletResponse.SC_NOT_FOUND,
                  "Tài nguyên không tồn tại");

// Gửi redirect
response.sendRedirect("https://example.com");
response.sendRedirect("/app/login");
```

```
// Lấy output stream
PrintWriter out = response.getWriter();
ServletOutputStream sos = response.getOutputStream(); // cho binary

// Set header
response.setHeader("Content-Disposition",
    "attachment; filename=file.pdf");
response.addHeader("Custom-Header", "value");

// Set cookie
Cookie cookie = new Cookie("username", "john");
cookie.setMaxAge(7 * 24 * 60 * 60); // 7 ngày
response.addCookie(cookie);
```

Ví dụ thực tế:

```
protected void doPost(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

    String action = request.getParameter("action");

    if ("delete".equals(action)) {
        // Xóa và chuyển hướng
        response.sendRedirect("/app/list");
    } else if ("download".equals(action)) {
        // Download file
        response.setContentType("application/pdf");
        response.setHeader("Content-Disposition",
            "attachment; filename=report.pdf");
        // Ghi dữ liệu file
    } else {
        // Lỗi
        response.sendError(HttpServletResponse.SC_BAD_REQUEST);
    }
}
```

Tóm tắt Session 3

Khái niệm	Giải thích
Servlet	Class Java chạy trên server, xử lý HTTP request
HttpServlet	Class cha cơ bản cho tất cả Servlet HTTP
doGet()	Phương thức xử lý GET request
doPost()	Phương thức xử lý POST request
HttpServletRequest	Object chứa dữ liệu request từ client

Khái niệm	Giải thích
HttpServletResponse	Object để gửi response về client

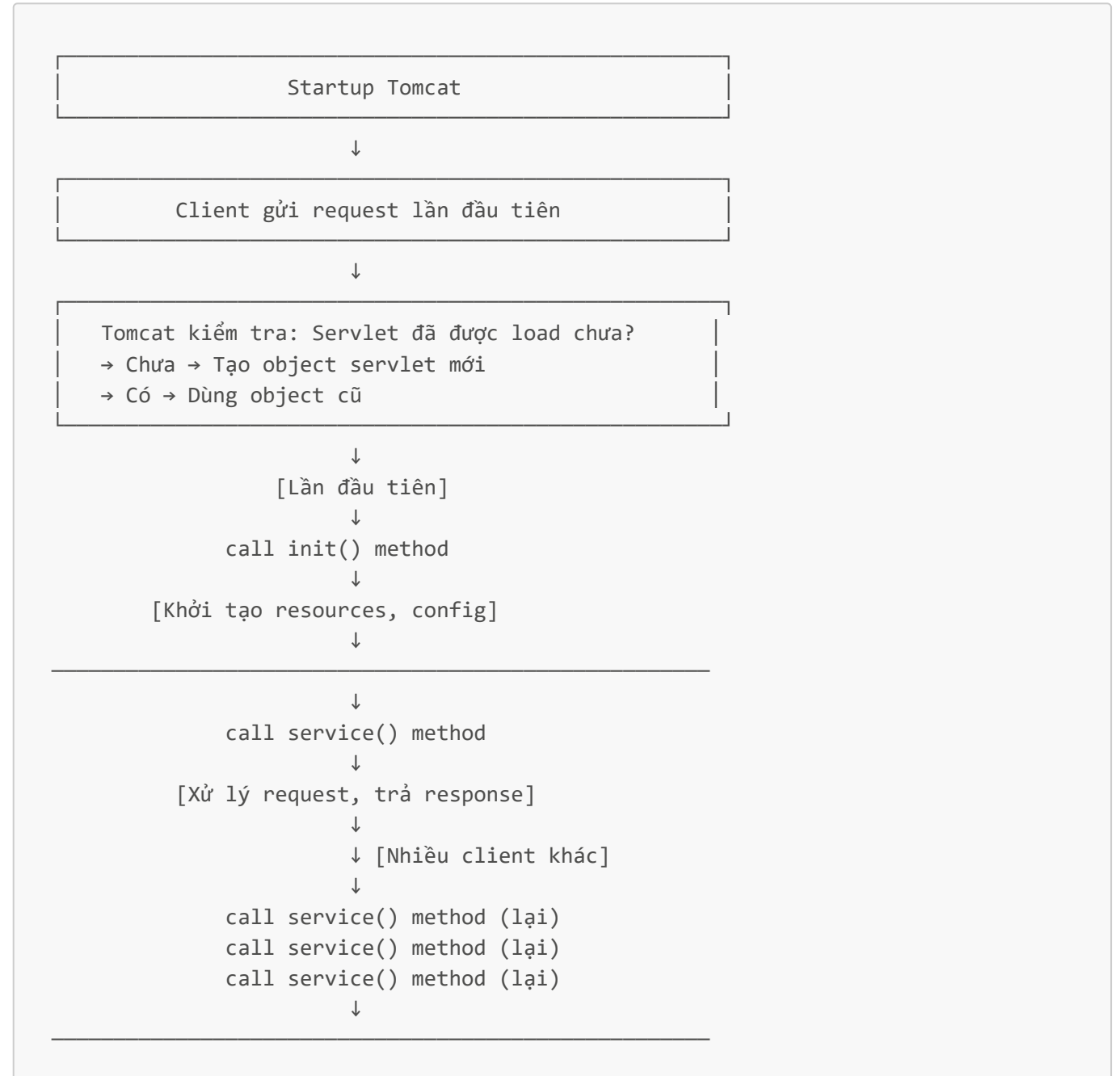
SESSION 4: Servlet Lifecycle

4.1 Vòng đời Servlet - Khái niệm chi tiết

Vòng đời Servlet gồm 3 giai đoạn:

- 1. **Initialization (Khởi tạo)** - gọi `init()`
- 2. **Service (Phục vụ)** - gọi `service()` → `doGet()/doPost()`
- 3. **Destruction (Tiêu hủy)** - gọi `destroy()`

Sơ đồ chi tiết



↓
[Khi Tomcat shutdown hoặc deploy lại]
↓
call destroy() method
↓
[Giải phóng resources, lưu state]
↓
Servlet object bị xóa khỏi memory

4.2 Phương thức init()

Mục đích: Khởi tạo Servlet, chuẩn bị resources cần thiết.

Khi nào gọi:

- Lần đầu tiên client gửi request
- Chỉ gọi MỘT LẦN duy nhất trong vòng đời Servlet

Ví dụ:

```
import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;

public class DatabaseServlet extends HttpServlet {

    private Connection dbConnection;
    private ServletConfig config;

    @Override
    public void init(ServletConfig config) throws ServletException {
        // Gọi init của class cha
        super.init(config);

        // Lưu config
        this.config = config;

        try {
            // Khởi tạo connection database
            Class.forName("com.mysql.jdbc.Driver");
            dbConnection = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/mydb",
                "root",
                "password"
            );
            System.out.println("✓ Database connection initialized");
        } catch (Exception e) {
            System.err.println("X Database connection failed");
            throw new ServletException(e);
        }
    }
}
```

```
}

// Có thể truy cập dbConnection trong doGet/doPost
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response) {
    // Dùng dbConnection ở đây
}
}
```

Các công việc thường làm trong init():

- Kết nối database
- Load configuration (từ web.xml)
- Khởi tạo connection pool
- Load file config bên ngoài
- Khởi tạo các object cần thiết

4.3 Phương thức service()

Mục đích: Xử lý HTTP Request từ client.

Khi nào gọi:

- Mỗi lần client gửi request
- Có thể gọi NHIỀU LẦN (hàng ngàn lần)

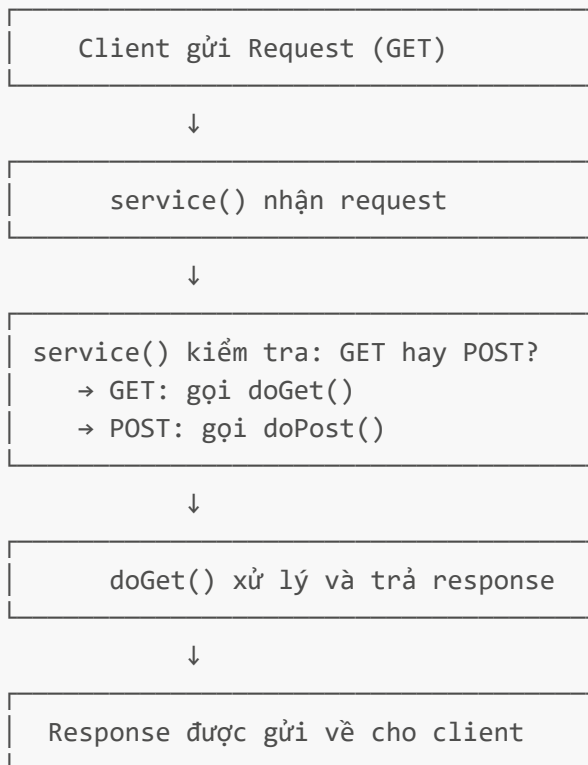
Hoạt động của service():

```
// Mã nội bộ của HttpServlet (bạn không cần viết)
protected void service(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {

    String method = request.getMethod();

    if ("GET".equals(method)) {
        doGet(request, response);
    }
    else if ("POST".equals(method)) {
        doPost(request, response);
    }
    else if ("PUT".equals(method)) {
        doPut(request, response);
    }
    else if ("DELETE".equals(method)) {
        delete(request, response);
    }
    // ... v.v.
}
```

Ở trong `service()`, nó sẽ gọi `doGet()`, `doPost()`, v.v. tùy theo HTTP method.



4.4 Phương thức `destroy()`

Mục đích: Dọn dẹp, giải phóng resources trước khi Servlet bị xóa.

Khi nào gọi:

- Khi Tomcat shutdown
- Khi deploy lại ứng dụng
- Chỉ gọi MỘT LẦN duy nhất

Ví dụ:

```
public class DatabaseServlet extends HttpServlet {

    private Connection dbConnection;

    @Override
    public void init(ServletConfig config) throws ServletException {
        // Khởi tạo connection
        try {
            // ... (mã khởi tạo)
            dbConnection = DriverManager.getConnection(...);
        } catch (Exception e) {
            throw new ServletException(e);
        }
    }
}
```

```

    }

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response) {
        // Sử dụng dbConnection
    }

    @Override
    public void destroy() {
        // Giải phóng connection
        try {
            if (dbConnection != null && !dbConnection.isClosed()) {
                dbConnection.close();
                System.out.println("✓ Database connection closed");
            }
        } catch (SQLException e) {
            System.err.println("X Error closing connection: " + e);
        }
    }
}

```

Các công việc thường làm trong destroy():

- Đóng database connection
- Lưu dữ liệu tạm thời
- Giải phóng tài nguyên
- Ghi log finalizing
- Cleanup file tạm

4.5 Ví dụ thực tế: Servlet Lifecycle Hoàn chỉnh

```

import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

public class LifecycleServlet extends HttpServlet {

    private int requestCount = 0;

    // 1. INITIALIZATION
    @Override
    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        System.out.println("♀ [INIT] LifecycleServlet đang khởi tạo...");
        System.out.println("   - Servlet name: " + config.getServletName());
    }
}

```

```

        System.out.println("    - Servlet context: " + config.getServletContext());
        System.out.println("✓ [INIT] Khởi tạo hoàn thành");
        requestCount = 0;
    }

    // 2. SERVICE - xử lý request
    @Override
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        requestCount++;

        System.out.println("☺ [SERVICE] Request #" + requestCount);
        System.out.println("    - Method: " + request.getMethod());
        System.out.println("    - URI: " + request.getRequestURI());
        System.out.println("    - Client IP: " + request.getRemoteAddr());

        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<head><title>Servlet Lifecycle</title></head>");
        out.println("<body>");
        out.println("<h1>Xin chào! Đây là request thứ " + requestCount + "</h1>");
        out.println("<p>Servlet đã xử lý " + requestCount + " request</p>");
        out.println("</body>");
        out.println("</html>");
    }

    // 3. DESTRUCTION
    @Override
    public void destroy() {
        System.out.println("☺ [DESTROY] LifecycleServlet đang tiêu hủy...");
        System.out.println("    - Tổng request nhận được: " + requestCount);
        System.out.println("✓ [DESTROY] Tiêu hủy hoàn thành");
    }
}

```

Output console khi chạy:

```

☺ [INIT] LifecycleServlet đang khởi tạo...
  - Servlet name: LifecycleServlet
  - Servlet context: org.apache.catalina.core.ApplicationContextFacade@abc123
✓ [INIT] Khởi tạo hoàn thành

[Client gửi request lần 1]
☺ [SERVICE] Request #1
  - Method: GET
  - URI: /app/lifecycle
  - Client IP: 127.0.0.1

```

```
[Client gửi request lần 2]
🕒 [SERVICE] Request #2
  - Method: GET
  - URI: /app/lifecycle
  - Client IP: 127.0.0.1

[Tomcat shutdown]
🕒 [DESTROY] LifecycleServlet đang tiêu hủy...
  - Tổng request nhận được: 2
✓ [DESTROY] Tiêu hủy hoàn thành
```

🔗 Tóm tắt Session 4

Giai đoạn	Phương thức	Gọi bao nhiêu lần	Mục đích
Initialization	init()	1 lần	Khởi tạo resources
Service	service()	Nhiều lần	Xử lý request
Service	doGet()/doPost()	Nhiều lần	Xử lý theo method
Destruction	destroy()	1 lần	Giải phóng resources

🍪 SESSION 5: Session & Cookies

5.1 Session Management Basics - Vì sao cần Session?

Vấn đề: HTTP không nhớ trạng thái (Stateless)

```
Request 1 (Client A):
GET /profile HTTP/1.1
→ Servlet: "Xin chào, ai vậy?"
→ Không biết client là ai

Response:
Vui lòng đăng nhập trước

---

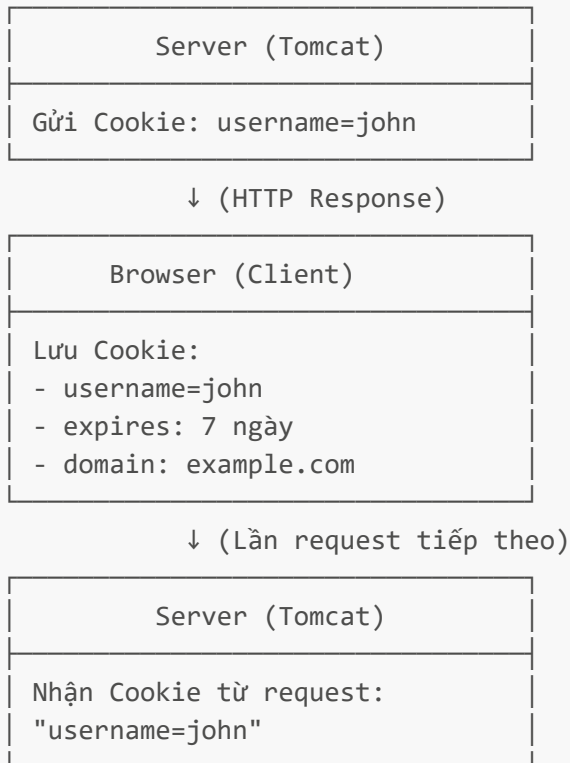
Request 2 (Client A):
GET /profile HTTP/1.1      (request khác)
→ Servlet: "Xin chào, ai vậy?"
→ Không nhớ request trước đó
→ Lại báo cần đăng nhập
```

Giải pháp: Dùng Session hoặc Cookie để lưu thông tin client

5.2 Cookie - Lưu trữ ở Client

Cookie là gì?

Cookie là một tập tin nhỏ lưu ở trình duyệt (client) của người dùng.



Tạo Cookie

```
// Tạo cookie
Cookie cookie = new Cookie("username", "john");

// Set hạn sử dụng (giờ)
// 7 ngày = 7 * 24 * 60 * 60 = 604800 giây
cookie.setMaxAge(7 * 24 * 60 * 60);

// Set domain
cookie.setDomain("example.com");

// Set path
cookie.setPath("/");

// Set secure (chỉ dùng với HTTPS)
cookie.setSecure(true);

// Set httpOnly (không cho JavaScript truy cập)
cookie.setHttpOnly(true);
```

```
// Gửi cookie về client
response.addCookie(cookie);
```

Ví dụ thực tế - Lưu username sau đăng nhập:

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");

    // Kiểm tra login (đơn giản, thực tế phải hash password)
    if ("admin".equals(username) && "123".equals(password)) {

        // Tạo session và lưu user info
        HttpSession session = request.getSession();
        session.setAttribute("username", username);

        // Tạo cookie "Remember Me"
        if ("on".equals(request.getParameter("remember"))) {
            Cookie cookie = new Cookie("username", username);
            cookie.setMaxAge(30 * 24 * 60 * 60); // 30 ngày
            response.addCookie(cookie);
        }

        response.sendRedirect("/app/home");
    } else {
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED);
    }
}
```

Đọc Cookie

```
protected void doGet(HttpServletRequest request,
                     HttpServletResponse response)
    throws ServletException, IOException {

    String rememberedUser = null;

    // Lấy tất cả cookies
    Cookie[] cookies = request.getCookies();

    if (cookies != null) {
        // Duyệt và tìm cookie "username"
        for (Cookie cookie : cookies) {
            if ("username".equals(cookie.getName())) {
                rememberedUser = cookie.getValue();
                break;
            }
        }
    }
}
```

```
    }  
  }  
}  
  
response.setContentType("text/html; charset=UTF-8");  
PrintWriter out = response.getWriter();  
out.println("<h1>Login Page</h1>");  
  
if (rememberedUser != null) {  
    out.println("<input type='text' value='" + rememberedUser + "'/>");  
}  
out.close();  
}
```

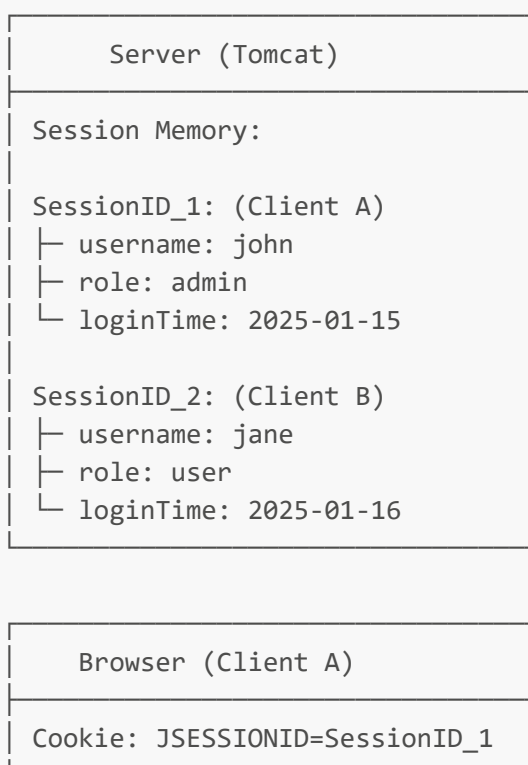
Xóa Cookie

```
// Tạo cookie cùng tên, nhưng maxAge = 0  
Cookie cookie = new Cookie("username", "");  
cookie.setMaxAge(0); // Xóa ngay  
response.addCookie(cookie);
```

5.3 Session - Lưu trữ ở Server

Session là gì?

Session là một vùng bộ nhớ trên server lưu thông tin của mỗi client.



Quy trình:

1. Client gửi request → Server
2. Server tạo Session object, lưu vào bộ nhớ
3. Server gửi SessionID qua Cookie về Client
4. Client lưu Cookie này
5. Lần request tiếp theo, Client gửi Cookie này kèm theo
6. Server kiểm tra SessionID, lấy thông tin từ memory

Tạo và sử dụng Session

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");

    if ("admin".equals(username) && "123".equals(password)) {
        // ✓ Đăng nhập thành công

        // Lấy session (nếu không có thì tạo mới)
        HttpSession session = request.getSession(true);

        // Lưu dữ liệu vào session
        session.setAttribute("username", username);
        session.setAttribute("role", "admin");
        session.setAttribute("loginTime", new Date());

        // Set thời gian hết hạn (30 phút)
        session.setMaxInactiveInterval(30 * 60);

        response.sendRedirect("/app/home");
    } else {
        // X Sai username/password
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED);
    }
}
```

Đọc dữ liệu từ Session

```
protected void doGet(HttpServletRequest request,
                     HttpServletResponse response)
    throws ServletException, IOException {

    // Lấy session (không tạo mới nếu không có)
    HttpSession session = request.getSession(false);
```

```
response.setContentType("text/html; charset=UTF-8");
PrintWriter out = response.getWriter();

if (session != null) {
    // Lấy dữ liệu từ session
    String username = (String) session.getAttribute("username");
    String role = (String) session.getAttribute("role");

    if (username != null) {
        out.println("<h1>Xin chào " + username + "!</h1>");
        out.println("<p>Quyền: " + role + "</p>");
        return;
    }
}

// Nếu không có session → chưa đăng nhập
out.println("<h1>Vui lòng đăng nhập</h1>");
out.println("<a href='/app/login'>Đến trang đăng nhập</a>");
out.close();
}
```

Xóa Session

```
protected void doGet(HttpServletRequest request,
                     HttpServletResponse response)
    throws ServletException, IOException {

    HttpSession session = request.getSession(false);

    if (session != null) {
        // Xóa session
        session.invalidate();
    }

    response.sendRedirect("/app/login");
}
```

5.4 Cookie vs Session

Yếu tố	Cookie	Session
Lưu ở đâu	Client (Browser)	Server (Memory)
Dung lượng	~4KB	Không giới hạn
An toàn	Thấp (dễ bị sửa)	Cao hơn (dữ liệu ở server)
Tốc độ	Nhanh (lấy từ client)	Chậm hơn (truy cập server)
Khi nào xóa	Hết hạn hoặc xóa thủ công	Session timeout

Yếu tố	Cookie	Session
Ví dụ dùng	Remember me, language	Login info, cart items

Tóm tắt Session 5

Khái niệm	Chi tiết
Cookie	Tập tin nhỏ lưu ở client, gửi lại mỗi request
Session	Vùng bộ nhớ server lưu info client
SessionID	Mã định danh duy nhất cho mỗi session
HttpSession	Object để quản lý session trong Java
getAttribute()	Lấy dữ liệu từ session
setAttribute()	Lưu dữ liệu vào session
invalidate()	Xóa session

SESSION 6: Deploy First Servlet

6.1 Chuẩn bị môi trường

Yêu cầu cần có:

1. **JDK** (Java Development Kit) - để biên dịch Java
2. **Apache Tomcat** - để chạy servlet
3. **IDE** (IntelliJ IDEA, Eclipse, VS Code) - để code
4. **Maven** hoặc **Gradle** (tùy chọn) - quản lý project

Kiểm tra Java đã cài đặt

```
java -version
javac -version
```

6.2 Download và cài đặt Apache Tomcat

Bước 1: Download Tomcat

Truy cập: <https://tomcat.apache.org/>

Chọn **Tomcat 10** (hoặc phiên bản mới nhất)

```
tomcat-10.1.x.zip (Windows) hoặc .tar.gz (Linux)
```

Bước 2: Giải nén

```
# Windows
C:\path\to\apache-tomcat-10.1.x\

# Linux
/opt/apache-tomcat-10.1.x/
```

Bước 3: Cấu hình biến môi trường (tùy chọn)

```
# Windows - thêm vào Path
CATALINA_HOME=C:\apache-tomcat-10.1.x

# Linux
export CATALINA_HOME=/opt/apache-tomcat-10.1.x
export PATH=$PATH:$CATALINA_HOME/bin
```

Bước 4: Khởi động Tomcat

Windows:

```
C:\apache-tomcat-10.1.x\bin\startup.bat
```

Linux/Mac:

```
/opt/apache-tomcat-10.1.x/bin/startup.sh
```

Truy cập: <http://localhost:8080>

Nếu thấy trang Tomcat → ✓ Cài đặt thành công

6.3 Tạo Project Servlet đầu tiên

Cách 1: Dùng IDE (IntelliJ IDEA)

Bước 1: Tạo project

- File → New → Project
- Chọn "Java"

- Chọn JDK → Next

Bước 2: Cấu hình Web

- Chọn "Web Application"
- Tomcat: Chọn đường dẫn Tomcat
- Next → Finish

Bước 3: Tạo Servlet

```
src/  
├ com.example/  
│   └ servlets/  
│       └ HelloServlet.java  
webapp/  
├ WEB-INF/  
│   └ web.xml  
└ index.html
```

Cách 2: Tạo thủ công

Bước 1: Tạo thư mục

```
mkdir MyWebApp  
cd MyWebApp  
mkdir -p src/main/java/com/example/servlets  
mkdir -p src/main/webapp/WEB-INF
```

Bước 2: Tạo HelloServlet.java

```
// src/main/java/com/example/servlets/HelloServlet.java  
package com.example.servlets;  
  
import javax.servlet.ServletException;  
import javax.servlet.annotation.WebServlet;  
import javax.servlet.http.HttpServlet;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import java.io.IOException;  
import java.io.PrintWriter;  
  
@WebServlet("/hello")  
public class HelloServlet extends HttpServlet {  
  
    @Override  
    protected void doGet(HttpServletRequest request,  
                          HttpServletResponse response)  
        throws ServletException, IOException {
```

```
response.setContentType("text/html; charset=UTF-8");
PrintWriter out = response.getWriter();

out.println("<html>");
out.println("<head>");
out.println("  <title>Hello Servlet</title>");
out.println("</head>");
out.println("<body>");
out.println("  <h1>Xin chào! Đây là servlet đầu tiên của tôi 🐼</h1>");
out.println("  <p>Tạm biệt HTTP Stateless!</p>");
out.println("</body>");
out.println("</html>");

out.close();
}
```

Bước 3: Tạo web.xml

```
<!-- src/main/webapp/WEB-INF/web.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0"
  xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">

  <!-- Nếu không dùng @WebServlet annotation, định nghĩa ở đây -->

  <display-name>My First Web Application</display-name>

</web-app>
```

6.4 Biên dịch và Deploy

Cách 1: Dùng IDE (dễ nhất)

1. Mở project trong IDE
2. Build project: Build → Build Project
3. Run project: Run → Run 'Tomcat 10'
4. Truy cập: <http://localhost:8080/MyWebApp/hello>

Cách 2: Dùng Command Line

Bước 1: Biên dịch

```
mkdir -p build/classes
javac -cp "C:\apache-tomcat-10.1.x\lib\*" ^
    -d build/classes ^
    src/main/java/com/example/servlets/HelloServlet.java
```

Bước 2: Tạo WAR file

```
mkdir -p build/MyWebApp/WEB-INF/classes
copy build/classes\*.class build/MyWebApp/WEB-INF/classes/
copy src/main/webapp/WEB-INF/web.xml build/MyWebApp/WEB-INF/

cd build
jar cvf MyWebApp.war MyWebApp/
```

Bước 3: Deploy

Copy `MyWebApp.war` vào: `C:\apache-tomcat-10.1.x\webapps\`

Tomcat sẽ tự động giải nén và chạy.

Bước 4: Kiểm tra

Truy cập: `http://localhost:8080/MyWebApp/hello`

6.5 Kiến trúc Project

```
MyWebApp/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/example/servlets/
│   │   │   │   └── HelloServlet.java
│   │   └── webapp/
│   │       ├── WEB-INF/
│   │       │   ├── web.xml
│   │       ├── index.html
│   │       ├── style/
│   │       │   └── style.css
│   │       └── js/
│   │           └── script.js
│   └── build/
│       └── MyWebApp.war
├── pom.xml (nếu dùng Maven)
└── README.md
```

6.6 Tomcat Directory Structure

```
apache-tomcat-10.1.x/
├── bin/
│   ├── startup.sh (startup.bat)
│   ├── shutdown.sh (shutdown.bat)
│   └── ...
├── conf/
│   ├── server.xml (Cấu hình server)
│   ├── web.xml (Cấu hình mặc định)
│   └── context.xml
├── lib/
│   └── (JAR files)
├── logs/
│   ├── catalina.out (Server log)
│   └── ...
├── webapps/
│   ├── ROOT/ (Ứng dụng mặc định: http://localhost:8080/)
│   ├── manager/ (Quản lý app)
│   ├── examples/ (Ví dụ)
│   ├── MyWebApp.war (Ứng dụng của bạn)
│   └── MyWebApp/ (Giải nén từ WAR)
└── temp/
```

WAR File là gì?

WAR = Web Archive

```
MyWebApp.war
└─ ↓ (Tomcat tự động giải nén)
MyWebApp/
├── WEB-INF/
│   ├── classes/          (Bytecode .class)
│   ├── lib/              (JAR files)
│   └── web.xml            (Cấu hình)
└── (Các file static: HTML, CSS, JS)
```

Tóm tắt Session 6

Bước	Chi tiết
1. Cài JDK	Để biên dịch Java
2. Cài Tomcat	Web server chạy servlet
3. Tạo Servlet	Viết class kế thừa HttpServlet
4. Biên dịch	javac hoặc IDE

Bước	Chi tiết
5. Tạo WAR	Gói lại dạng WAR file
6. Deploy	Copy WAR vào webapps/
7. Kiểm tra	Truy cập URL servlet

SESSION 7: Mini Project - Simple "Hello World" Web App with Form Input

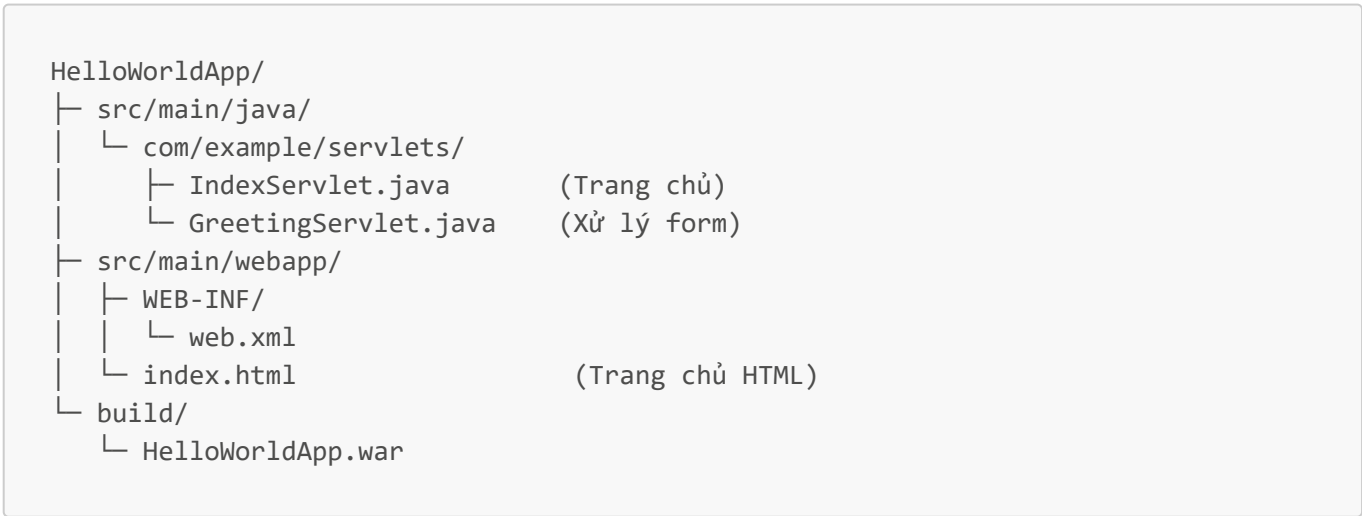
7.1 Yêu cầu dự án

Tạo ứng dụng web đơn giản:

- 1. **Trang chủ** - hiển thị form nhập tên
- 2. **Form submission** - gửi dữ liệu qua POST
- 3. **Xử lý & hiển thị** - Servlet xử lý và in ra lời chào personalized



7.2 Cấu trúc dự án



7.3 Code - Phần 1: Trang chủ (index.html)

```
<!-- src/main/webapp/index.html -->
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ứng dụng Hello World đầu tiên</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 600px;
      margin: 100px auto;
      padding: 20px;
      background-color: #f0f0f0;
    }

    .container {
      background-color: white;
      border-radius: 8px;
      padding: 30px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }

    h1 {
      color: #333;
      text-align: center;
    }

    .form-group {
      margin: 15px 0;
    }

    label {
      display: block;
      margin-bottom: 5px;
      color: #666;
      font-weight: bold;
    }

    input[type="text"],
    input[type="email"],
    select {
      width: 100%;
      padding: 10px;
      border: 1px solid #ddd;
      border-radius: 4px;
      box-sizing: border-box;
      font-size: 14px;
    }

    input[type="text"]:focus,
    input[type="email"]:focus,
    select:focus {
```

```

        outline: none;
        border-color: #4CAF50;
        box-shadow: 0 0 5px rgba(76, 175, 80, 0.3);
    }

    button {
        width: 100%;
        padding: 12px;
        background-color: #4CAF50;
        color: white;
        border: none;
        border-radius: 4px;
        font-size: 16px;
        cursor: pointer;
        font-weight: bold;
        transition: background-color 0.3s;
    }

    button:hover {
        background-color: #45a049;
    }

    .info {
        background-color: #e3f2fd;
        border-left: 4px solid #2196F3;
        padding: 15px;
        margin-top: 20px;
        border-radius: 4px;
    }
}
</style>
</head>
<body>
    <div class="container">
        <h1>👋 Ứng dụng Hello World đầu tiên</h1>

        <form action="greeting" method="POST">
            <div class="form-group">
                <label for="name">Nhập tên của bạn:</label>
                <input type="text"
                    id="name"
                    name="name"
                    placeholder="Ví dụ: Nhân, Phúc, Hải..."
                    required />
            </div>

            <div class="form-group">
                <label for="country">Bạn đến từ nước nào?</label>
                <select id="country" name="country" required>
                    <option value="">-- Chọn quốc gia --</option>
                    <option value="Vietnam">Việt Nam</option>
                    <option value="Thailand">Thái Lan</option>
                    <option value="USA">Hoa Kỳ</option>
                    <option value="Japan">Nhật Bản</option>
                    <option value="Korea">Hàn Quốc</option>
                </select>
            </div>
        </form>
    </div>

```

```
        <option value="Singapore">Singapore</option>
        <option value="Other">Khác</option>
    </select>
</div>

    <button type="submit">✦ Gửi</button>
</form>

<div class="info">
    <strong>i Ghi chú:</strong>
    <p>Nhập tên của bạn và quốc gia, sau đó nhấn nút "Gửi".</p>
    <p>Servlet sẽ xử lý dữ liệu của bạn và hiển thị lời chào personalized.
</p>
</div>
</div>
</body>
</html>
```

7.4 Code - Phần 2: GreetingServlet (Xử lý form)

```
// src/main/java/com/example/servlets/GreetingServlet.java
package com.example.servlets;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

@WebServlet("/greeting")
public class GreetingServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        // 1. Lấy dữ liệu từ form
        String name = request.getParameter("name");
        String country = request.getParameter("country");

        // 2. Validate dữ liệu
        if (name == null || name.trim().isEmpty()) {
            name = "Bạn";
        } else {
            name = name.trim();
        }
    }
}
```

```
}

if (country == null || country.isEmpty()) {
    country = "nước không xác định";
}

// 3. Xử lý logic (có thể lưu DB, ghi log, v.v.)
int nameLength = name.length();
LocalDateTime now = LocalDateTime.now();
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy
HH:mm:ss");
String currentTime = now.format(formatter);

// 4. Xác định xưng hô
String greeting = getGreeting(now.getHour());

// 5. Set response type và encoding
response.setContentType("text/html; charset=UTF-8");

// 6. Ghi response HTML
PrintWriter out = response.getWriter();

out.println("<!DOCTYPE html>");
out.println("<html lang=\"vi\">");
out.println("<head>");
out.println("  <meta charset=\"UTF-8\">");
out.println("  <meta name=\"viewport\" content=\"width=device-width,
initial-scale=1.0\">");
out.println("  <title>Kết quả</title>");
out.println("  <style>");
out.println("    body {");
out.println("      font-family: Arial, sans-serif;");
out.println("      max-width: 600px;");
out.println("      margin: 100px auto;");
out.println("      padding: 20px;");
out.println("      background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);");
out.println("      min-height: 100vh;");
out.println("    }");
out.println("  .container {");
out.println("    background-color: white;");
out.println("    border-radius: 10px;");
out.println("    padding: 40px;");
out.println("    box-shadow: 0 10px 30px rgba(0,0,0,0.3);");
out.println("    text-align: center;");
out.println("  }");
out.println("  .greeting {");
out.println("    font-size: 28px;");
out.println("    color: #667eea;");
out.println("    margin-bottom: 20px;");
out.println("    font-weight: bold;");
out.println("  }");
out.println("  .name {");
out.println("    font-size: 36px;");
```

```

        out.println("        color: #764ba2;");
        out.println("        margin: 10px 0;");
        out.println("    }");
        out.println("    .info {");
        out.println("        background-color: #f5f5f5;");
        out.println("        padding: 15px;");
        out.println("        border-radius: 5px;");
        out.println("        margin-top: 20px;");
        out.println("        font-size: 14px;");
        out.println("        color: #666;");
        out.println("    }");
        out.println("    .info p {");
        out.println("        margin: 5px 0;");
        out.println("    }");
        out.println("    .button {");
        out.println("        display: inline-block;");
        out.println("        margin-top: 20px;");
        out.println("        padding: 12px 30px;");
        out.println("        background-color: #667eea;");
        out.println("        color: white;");
        out.println("        text-decoration: none;");
        out.println("        border-radius: 5px;");
        out.println("        font-weight: bold;");
        out.println("        transition: background-color 0.3s;");
        out.println("    }");
        out.println("    .button:hover {");
        out.println("        background-color: #764ba2;");
        out.println("    }");
        out.println("    </style>");
        out.println("</head>");
        out.println("<body>");
        out.println("    <div class=\"container\">");
        out.println("        <div class=\"greeting\"> + greeting + "</div>");
        out.println("        <div class=\"name\"> + name + "! 🐼</div>");
        out.println("        <p>Chúc bạn một ngày tuyệt vời!</p>");
        out.println("        <div class=\"info\">");
        out.println("            <p><strong>Quốc gia:</strong> " + country + "</p>");
        out.println("            <p><strong>Độ dài tên:</strong> " + nameLength + " ký");
        out.println("            <p><strong>Thời gian xử lý:</strong> " + currentTime +");
        out.println("            </div>");
        out.println("            <a href=\"/HelloWorldApp/\" class=\"button\">← Quay");
        out.println("            lại</a>");
        out.println("        </div>");
        out.println("</body>");
        out.println("</html>");

        out.close();
    }

```

@Override

```

protected void doGet(HttpServletRequest request,
    HttpServletResponse response)

```

```

        throws ServletException, IOException {

        // Nếu ai cố truy cập trực tiếp qua GET
        response.sendRedirect("/HelloWorldApp/");
    }

    /**
     * Phương thức hỗ trợ: Xác định lời chào dựa trên giờ
     */
    private String getGreeting(int hour) {
        if (hour >= 5 && hour < 12) {
            return "🌞 Chào buổi sáng";
        } else if (hour >= 12 && hour < 17) {
            return "🌤️ Chào buổi chiều";
        } else if (hour >= 17 && hour < 21) {
            return "🌃 Chào buổi tối";
        } else {
            return "🌙 Chào đêm";
        }
    }
}

```

7.5 Cấu hình web.xml

```

<!-- src/main/webapp/WEB-INF/web.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0"
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">

    <display-name>Hello World App</display-name>

    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>

</web-app>

```

7.6 Chạy dự án

Bước 1: Tạo WAR file

```

# Biên dịch
mkdir -p build/classes
javac -cp "C:\apache-tomcat-10.1.x\lib\*" ^

```

```
-d build/classes ^
src/main/java/com/example/servlets/GreetingServlet.java

# Tạo WAR
mkdir -p build/HelloWorldApp/WEB-INF/classes
copy build/classes\* build/HelloWorldApp/WEB-INF\classes\
copy src/main/webapp/WEB-INF/web.xml build/HelloWorldApp/WEB-INF\
copy src/main/webapp/index.html build/HelloWorldApp\

cd build
jar cvf HelloWorldApp.war HelloWorldApp/
```

Bước 2: Deploy

Copy `HelloWorldApp.war` vào `C:\apache-tomcat-10.1.x\webapps\`

Tomcat sẽ tự động deploy.

Bước 3: Kiểm tra

Truy cập: `http://localhost:8080/HelloWorldApp/`

7.7 Chức năng bổ sung (Tùy chọn)

Thêm Logging

```
protected void doPost(...) {
    String name = request.getParameter("name");

    // Ghi log
    System.out.println "[" + new Date() + "] Request từ: " + name);
}
```

Thêm Validation Server-side

```
if (name == null || name.trim().isEmpty()) {
    response.sendError(HttpServletResponse.SC_BAD_REQUEST,
        "Vui lòng nhập tên");
    return;
}

// Kiểm tra độ dài
if (name.length() > 50) {
    response.sendError(HttpServletResponse.SC_BAD_REQUEST,
        "Tên quá dài (tối đa 50 ký tự)");
    return;
}
```

Lưu dữ liệu vào file

```
import java.io.FileWriter;
import java.io.BufferedWriter;

String logFile = getServletContext().getRealPath("/logs/submissions.txt");
try (BufferedWriter writer = new BufferedWriter(new FileWriter(logFile, true))) {
    writer.write(name + " | " + country + " | " + new Date() + "\n");
}
```

Hệ thống học tập

Hiểu biết sau khi hoàn thành Mini Project:

☒ Tạo HTML form ☒ Xử lý POST request trong Servlet ☒ Validate dữ liệu ☒ Render HTML động ☒
Deploy WAR file ☒ Quản lý URL mapping

Hoàn thành Tuần 1!

Bạn đã học:

- ☒ JVM, Java EE, Servlet Container
- ☒ HTTP GET/POST, Status codes
- ☒ Servlet cơ bản, Request/Response
- ☒ Servlet Lifecycle (init, service, destroy)
- ☒ Session & Cookies
- ☒ Deploy Servlet trên Tomcat
- ☒ Mini Project thực tế

Bước tiếp theo: Tuần 2 - JSP Basics

Tài liệu tham khảo

Java EE & Servlet

- Oracle Java EE Documentation: <https://www.oracle.com/java/technologies/javaee/>
- Tomcat Official Documentation: <https://tomcat.apache.org/>
- Servlet API: <https://javaee.github.io/servlet-spec/>

Học thêm

- Java Servlet Tutorial: <https://www.javatpoint.com/servlet-tutorial>
- W3Schools HTTP: https://www.w3schools.com/whatis/whatis_http.asp
- RFC 2616 HTTP/1.1: <https://tools.ietf.org/html/rfc2616>

Code Examples

- [GitHub - Java Servlet Examples](#)
- [Eclipse IDE Project Examples](#)

Tài liệu được biên soạn bởi: Senior Java Developer Ngày cập nhật: 2025 Phiên bản: 1.0